qnx-buildfile-lang documentation 1.0.6

Table of Contents

Giovanni Vergine, verginegiovanni@gmail.com

Date: Sun Dec 28 18:12:46 EET 2025

1. qnx-buildfile-lang

QNX buildfiles play a central role in many embedded and automotive systems. As systems scale, buildfiles naturally increase in complexity, which makes early validation, consistency, and policy enforcement increasingly important.

`qnx-buildfile-lang` project provides:

- A complete Xtext grammar for QNX buildfiles, ensuring instant syntax validation, syntax highlighting and code completion inside Eclipse
- A reference executable jar for generic validation of buildfiles. Maven library that enables programmatic parsing, additinal opportunities for semantic validation, automation, and CI checks.
- A standalone Maven library that can be used to write your own java-based tooling, enabling programmatic parsing, additional semantic validation, automation, and CI checks.

This project was built to make QNX development smoother, more reliable, and more automatable for teams working on embedded or automotive platforms.

1.1. Key Features

`qnx-buildfile-lang` is a set of java-based tooling implemented using Xtext for parsing QNX Buildfiles

- Available as an Eclipse Plugin
  - Syntax highlighting
  - Real-time syntax validation
  - Better readability and safer editing
- Available as command line standalone executable jar
  - Enables parsing of buildfiles programmatically
  - Useful for addding validation to CI/CD pipelines
- Available as a standalone Java library
  - Integrate into custom tooling
  - Distributed via Maven Central

1.2. Project Status & Compatibility

- Minimum JRE required: 17

1.3. Resources

- Source code: qnx-buildfile-lang
- Issue tracker: qnx-buildfile-lang/issues

- License: Apache License Version 2.0, January 2004

2. The Eclipse plugin

Like most of the Xtext-based projects, an Eclipse Feature is available and is deployed to Maven. Follow the steps below to understand how to get it, install it and use it.

2.1. Installation

Download the Eclipse Repository as a ZIP from Maven Central. Look for `qnx.buildfile.lang.repository-{Version}.zip` in Maven Central

E.g. latest version `1.0.6` can be found at https://repo1.maven.org/maven2/io/github/gvergine/qnx.buildfile.lang.repository/1.0.6/

Once downloded, follow the usual steps to install it in Eclipse:

- Help → Install New Software → Add → Archive...
- Select the ZIP file and proceed with installation
- Restart Eclipse

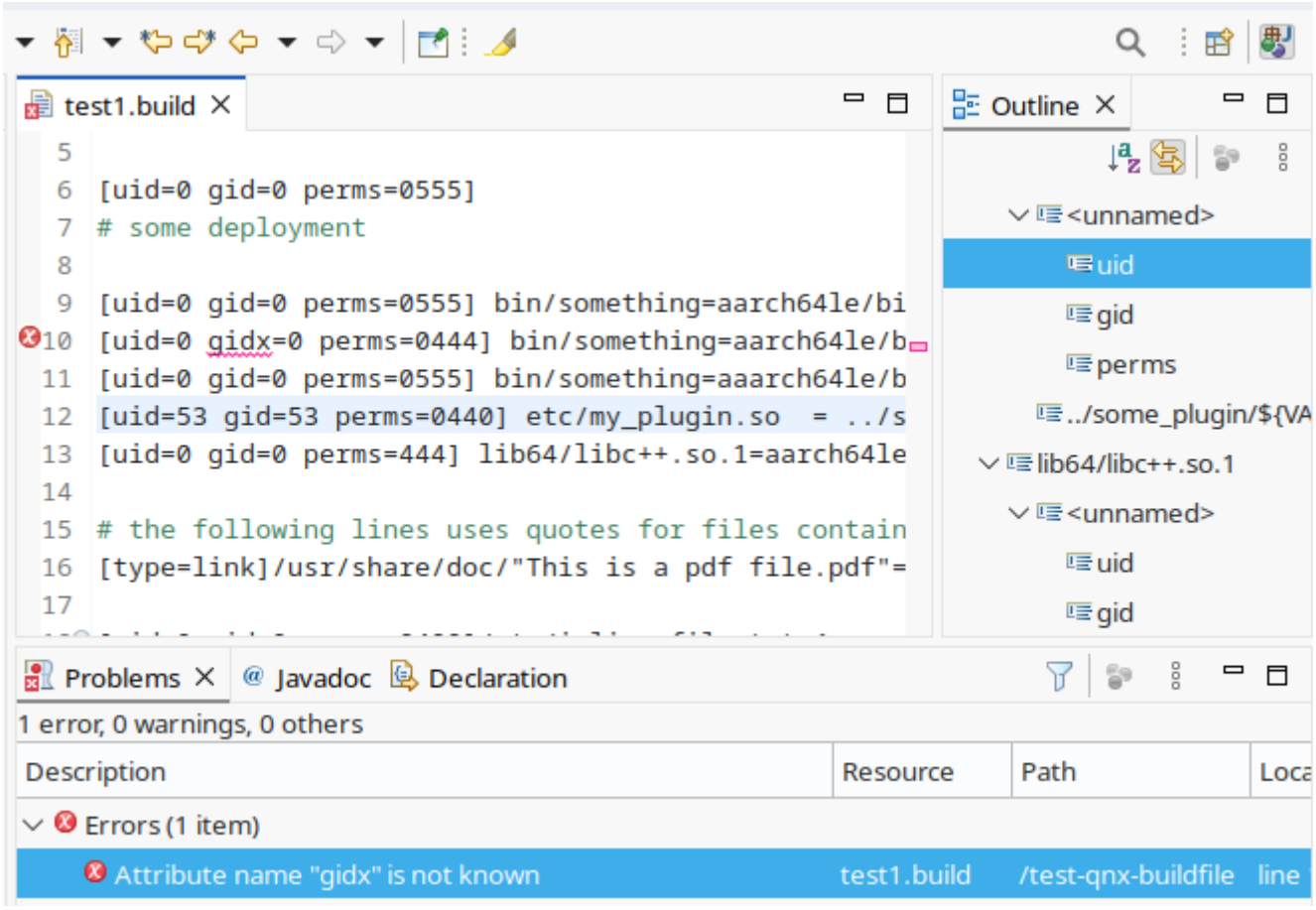
Even if Momentics is Eclipse-based, support for Momentics IDE is still work in progress.

2.2. Usage

Xtext Nature

When opening a QNX .build file for the first time in an Eclipse project, Eclipse will ask if to allow the Xtext Nature for such project. This is mandatory to enable the editor.

Syntax checking and highlighting should be available as in the following picture:


test1.build
5
6 [uid=0 gid=0 perms=0555]
7 # some deployment
8
9 [uid=0 gid=0 perms=0555] bin/something=aarch64le/bi
10 [uid=0 gidx=0 perms=0444] bin/something=aarch64le/b
11 [uid=0 gid=0 perms=0555] bin/something=aaarch64le/b
12 [uid=53 gid=53 perms=0440] etc/my_plugin.so = ../s
13 [uid=0 gid=0 perms=444] lib64/libc++.so.1=aarch64le
14
15 # the following lines uses quotes for files contain
16 [type=link]/usr/share/doc/"This is a pdf file.pdf"=
17
Outline
<unnamed>
uid
gid
perms
../some_plugin/${VA
lib64/libc++.so.1
<unnamed>
uid
gid
Problems
Javadoc
Declaration
1 error, 0 warnings, 0 others
Description
Resource
Path
Loca
Errors (1 item)
Attribute name "gidx" is not known
test1.build
/test-qnx-buildfile
line

3. The command line standalone executable jar

A ready-to-use executable jar for command line validation is available as well. This is mostly useful for integration in a script or in a CI/CD pipeline.

It supports the same validation rules of the Eclipse Plugin, but they are printed on the standard error, and the exit code will be 0 only if no errors are found.

3.1. Installation

Download the command line standalone executable jar from Maven Central. Look for `qnx.buildfile.lang.cli-{Version}-shaded.jar` in Maven Central

E.g. latest version `1.0.6` can be found at https://repo1.maven.org/maven2/io/github/gvergine/qnx.buildfile.lang.cli/1.0.6/

3.2. Usage

Java Version

Make sure you use a JRE version 17+

You can specify multiple files on the same command line, e.g:

```
java -jar qnx.buildfile.lang.cli-1.0.6-shaded.jar -i=<inputs>[,<inputs>...] [-i
=<inputs>[,<inputs>...]]...
  -i=<inputs>[,<inputs>...]
         buildfile(s)
```

This example shows a succesful run:

```
$ java -jar qnx.buildfile.lang.cli-1.0.6-shaded.jar -i path/to/first.build -i
path/to/second.build
QNX Buildfile Validator version 1.0.6
Processing path/to/first.build
Processing path/to/second.build
Done - 0 failures
$ echo $?
0
```

Errors and warnings will be reported on standard error, and exit code will be 1 if any error occourred:

```
$ java -jar qnx.buildfile.lang.cli-1.0.6-shaded.jar -i path/to/error.build
```

```
QNX Buildfile Validator version 1.0.6
Processing path/to/error.build
ERROR at path/to/error.build:11: Attribute name "gidh" is not known
Done - 1 failure
$ echo $?
1
```

4. Getting Started with the standalone Java library

The standalone Java library allows you to build your own java-based tooling. Typical use cases are: - extending the validation rules for enforcing additional policies - integrate with java-based environments (e.g. Tomcat, Java-based GUIs) - generation (e.g. xml or csv representation of the buildfile for further processing with non-java tools)

4.1. Installation

You can get the latest version via Maven Central:

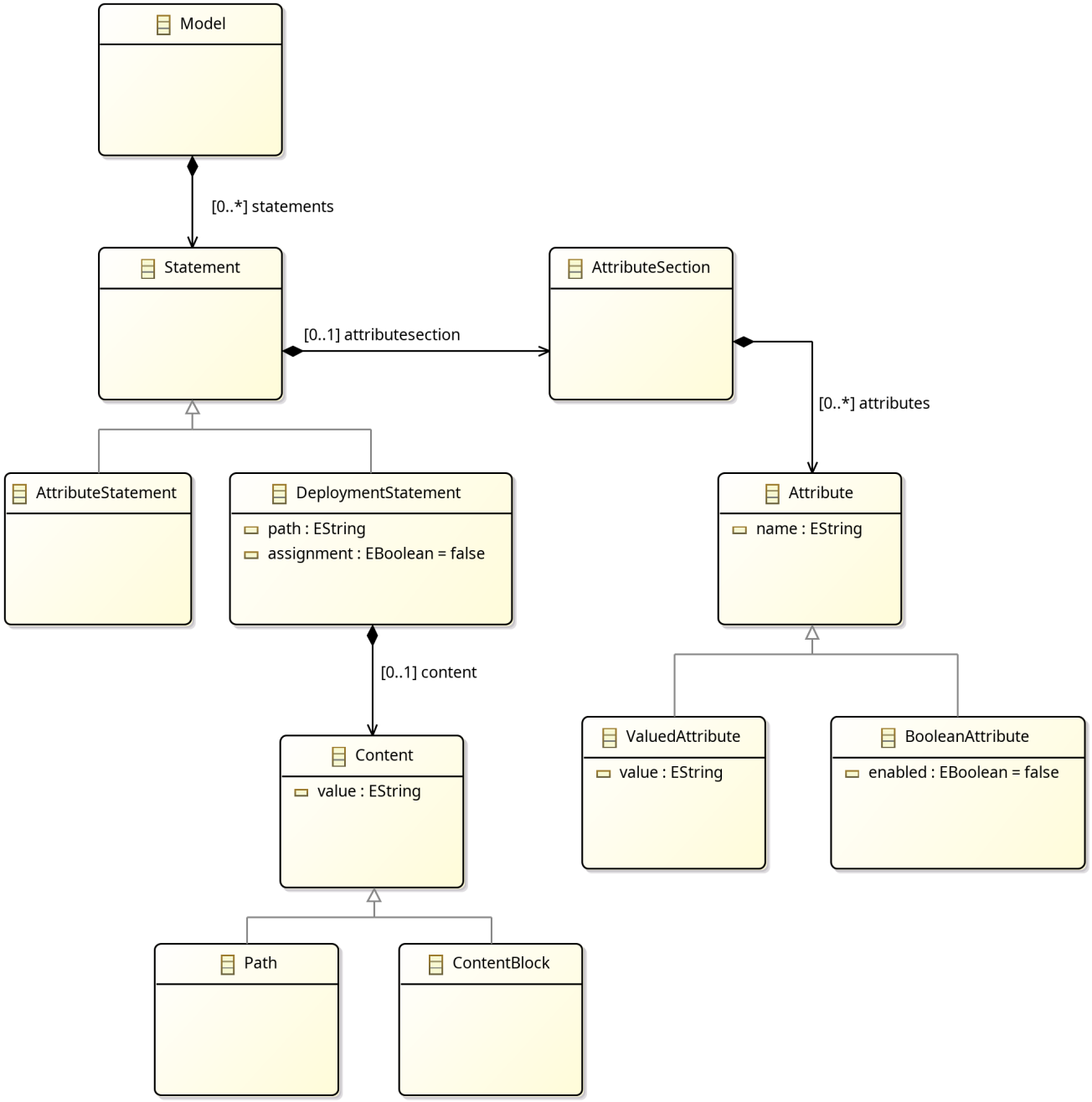
```
<dependency>
  <groupId>io.github.gvergine</groupId>
  <artifactId>qnx.buildfile.lang</artifactId>
  <version>1.0.6</version>
</dependency>
```

4.2. Key concepts to remember

When using a parser library, it is important first to understand the Abstract Syntax Tree that the parser will produce in memory. When the AST is known, will be easy to access any part of the model and to validate it or generate output accordingly.

There are multiple ways to see and remember the AST.

- If you are familiar with the Xtext grammar notation, you can see understand the AST by looking at it.
- Javadoc is generated and available via Maven as well
- Referring to the following class diagram:



4.3. Examples

4.3.1. Parsing programmatically:

```
import qnx.buildfile.lang.utils.Parser;
import qnx.buildfile.lang.utils.ParsingResult;

// ...

Parser parser = new Parser();
File file = new File(filename);
ParsingResult parseResult = parser.parse(file);

if (parseResult.hasErrors()) {
```

```
    // ...
```

4.3.2. Writing and using a custom validator:

```
import org.eclipse.xtext.validation.Check;
import qnx.buildfile.lang.attributes.AttributeKeywords;
import qnx.buildfile.lang.buildfileDSL.Attribute;
import qnx.buildfile.lang.buildfileDSL.BuildfileDSLPackage;
import qnx.buildfile.lang.validation.BuildfileDSLValidator;

public class CustomBuildfileDSLValidator extends BuildfileDSLValidator
{
    @Check
    public void checkAttributes(Attribute attribute)
    {
        if (!AttributeKeywords.ALL_ATTRIBUTE_KEYWORDS
                .contains(attribute.getName()))
        {
            error("Attribute name \"" + attribute.getName() + "\" is unknown",
                    BuildfileDSLPackage.Literals.ATTRIBUTE__NAME,
                    "invalidName");
        }
    }
}
```

For example, the following custom validator adds a warning for duplicate paths:

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import org.eclipse.xtext.validation.Check;

import qnx.buildfile.lang.buildfileDSL.BuildfileDSLPackage;
import qnx.buildfile.lang.buildfileDSL.DeploymentStatement;
import qnx.buildfile.lang.buildfileDSL.Model;
import qnx.buildfile.lang.utils.Walker;
import qnx.buildfile.lang.utils.Walker.IWalker;
import qnx.buildfile.lang.validation.BuildfileDSLValidator;

public class CustomBuildfileDSLValidator extends BuildfileDSLValidator {

    private final static Walker walker = new Walker();

    @Check
    public void checkDuplicates(Model model) {
        Map<String,List<DeploymentStatement>> duplicates = new HashMap<>();
```

```
        walker.walk(model, new IWalker() {
            @Override
            public void found(DeploymentStatement deploymentStatement)
            {
                String path = deploymentStatement.getPath();

                if (duplicates.containsKey(path))
                {
                    duplicates.get(path).add(deploymentStatement);
                }
                else
                {
                    List<DeploymentStatement> l = new ArrayList<>();
                    l.add(deploymentStatement);
                    duplicates.put(path, l);
                }

            };
        });

        duplicates.forEach((path, deployments) ->{
            if (deployments.size() > 1)
            {
                for (DeploymentStatement deployment : deployments)
                {
                    warning("Duplicate path " + path, deployment,
BuildfileDSLPackage.Literals.DEPLOYMENT_STATEMENT__PATH, "duplicatePath");
                }
            }

        });

    }
}
```

```
import qnx.buildfile.lang.utils.Parser;
import qnx.buildfile.lang.utils.ParsingResult;

// ...

Parser parser = new Parser(CustomBuildfileDSLValidator.class);
File file = new File(filename);
ParsingResult parseResult = parser.parse(file);

if (parseResult.hasErrors()) {
    // ...
```

4.3.3. Resolve the environment variables:

```
import qnx.buildfile.lang.utils.VariableSubstitutor;

// ...
final Map<String, String> envMap = System.getenv();
VariableSubstitutor vs = new VariableSubstitutor();
vs.substituteVariables(model, envMap);
```